

Agents Lab

Samarth Bhatnagar

September 2026

Contents

Contents

1	Task 0: Understand the Search Problem	2
2	Task 1: Plan the Agent	3
3	Task 2: Ask an LLM to Generate A*	5
4	Task 3: Test the Generated Program	6
5	Task 4: Inspect the A* Algorithm	9
6	Task 5: Compare A* with Blind Search	12
7	Task 6: Investigate the Heuristic	14
8	Task 7: Evaluate the LLM-Generated Agent	16
9	Final Reflection	18

1 Task 0: Understand the Search Problem

The warehouse navigation problem can be formulated as a search problem

$$P = (S, A, T, s_0, G, c),$$

where S is the set of states, A is the set of actions, T is the transition function, s_0 is the initial state, G is the set of goal states, and c is the cost function.

Search Problem Formulation

Component	Specification
State S	A state is the current position of the robot in the warehouse grid, represented by its row and column coordinates (r, c) corresponding to a free cell.
Actions A	The available actions are Up, Down, Left, and Right. Each action moves the robot by one grid cell in the corresponding direction.
Transition T	The transition function maps a state and a valid action to the resulting grid position. For example, moving Up from (r, c) results in $(r - 1, c)$. A transition is possible only when the resulting cell is within the warehouse and is not an obstacle.
Initial state s_0	The initial state is the position of the starting cell S in the warehouse grid.
Goal G	The goal state is the position of the cell G . A solution is reached when the robot's current state is the goal state.
Cost c	Every movement from one grid cell to an adjacent cell has a cost of 1. Therefore, the total cost of a path is equal to the number of moves in that path.

Table 1: Formulation of the warehouse navigation problem as a search problem.

(a) What information is necessary to specify a state?

A state is completely specified by the current row and column position of the robot in the warehouse grid:

$$s = (r, c).$$

The position must correspond to a valid, non-obstacle cell in the warehouse.

(b) What makes an action invalid?

An action is invalid if it would move the robot outside the boundaries of the warehouse or into a cell containing an obstacle, represented by **#**. Therefore, only movements to valid free cells are allowed.

(c) Is this a deterministic search problem?

Yes. The search problem is deterministic because each valid action performed from a given state always produces the same resulting state. There is no randomness or uncertainty in the movement of the robot.

(d) What would constitute a solution?

A solution is a sequence of valid actions that takes the robot from the initial state s_0 , corresponding to $\mathbf{S}$, to the goal state corresponding to $\mathbf{G}$, without entering any obstacle cells.

The cost of the solution is the number of movements in the path, since each movement has cost 1.

2 Task 1: Plan the Agent

Before using the Large Language Model, the search agent was designed by specifying how states, actions, the warehouse, the frontier, and path reconstruction would be represented. The design is independent of the implementation produced later by the LLM.

1. State Representation

A state will be represented in Python as a tuple containing the row and column coordinates of the robot:

$$(r, c)$$

For example, the position in row r and column c is represented as $(\mathbf{r}, \mathbf{c})$. This allows each grid position to be used directly as a search state.

2. Warehouse Representation

The warehouse will be represented as a list of strings, where each character corresponds to one grid cell:

- $\#$ represents an obstacle;
- $.$ represents a free cell;
- $\mathbf{S}$ represents the starting position;
- $\mathbf{G}$ represents the goal position.

This representation allows the program to determine whether a position is free or blocked by examining the corresponding character in the grid.

3. Determining Valid Actions

From each state, the agent will consider the four possible actions:

- Up: $(-1, 0)$
- Down: $(1, 0)$
- Left: $(0, -1)$
- Right: $(0, 1)$

An action will be considered valid if the resulting position is inside the warehouse boundaries and the corresponding grid cell is not an obstacle.

4. Recognising the Goal

The agent will recognise that it has reached the goal when the current state is equal to the position of $\mathbf{G}$.

$$(r, c) = (r_G, c_G)$$

When this condition is satisfied, the search terminates and the path to that state is returned as the solution.

5. Information Stored in the Frontier

The frontier will store states that have been discovered but have not yet been expanded. For each state, the search needs sufficient information to determine its priority and reconstruct the path.

For A* search, this includes the cost accumulated from the initial state, $g(n)$, and the estimated cost to the goal, $h(n)$. The evaluation function is then

$$f(n) = g(n) + h(n).$$

The frontier will therefore allow the search algorithm to select the state with the smallest evaluation value for expansion.

6. Path Reconstruction

To reconstruct the final path, the program will maintain information about the predecessor of each discovered state. When a state is reached, its predecessor is recorded.

Once the goal is reached, the program will start from the goal and follow the predecessor information backwards until the initial state is reached. The resulting sequence will then be reversed to obtain the path from $\mathbf{S}$ to $\mathbf{G}$.

Information Reported by the Program

When the search terminates, the program will report at least the following information:

- whether a solution was found;
- the solution path;
- the path length;
- the number of states expanded.

These measurements will also be used later to test the generated program and compare A* with blind search.

3 Task 2: Ask an LLM to Generate A*

After designing the search agent, the Large Language Model was used as an engineering assistant to translate the design into a Python implementation of A* search.

Prompt Used

The following prompt was given to the Large Language Model:

I am implementing a simple goal-based search agent in Python for the warehouse navigation problem.

The warehouse is represented as an ASCII grid. The robot starts at **S** and must reach **G**. The symbol **#** represents an obstacle and **.** represents a free cell. The robot can move Up, Down, Left, or Right, and every movement has a cost of 1.

Implement the A* search algorithm according to the following design:

- represent each grid position as a state using its row and column coordinates;
- represent the warehouse as a two-dimensional grid;
- determine valid actions by checking the grid boundaries and obstacle cells;
- recognise the goal when the current state is the position of **G**;
- maintain an appropriate frontier for A* search;
- calculate the path cost $g(n)$;
- use Manhattan distance as the heuristic:

$$h(n) = |x - x_G| + |y - y_G|;$$

- calculate the evaluation function

$$f(n) = g(n) + h(n);$$

- avoid unnecessarily expanding the same state repeatedly;
- store predecessor information so that the final path can be reconstructed when the goal is reached.

The program should use the following exact warehouse map:

```
#####
#S...#.....#
#...#.#.....#
#...#.#.....#
###.#.#.....#
#...#.#.....#
#.#.....#.#
#.#.....#.#
#.....#G#
#####
```

When the program terminates, it should report:

- whether a solution was found;
- the solution path;
- the path length;
- the number of states expanded.

Keep the implementation simple and well documented. Explain the main components of the A* implementation.

Run the generated program on the given warehouse map and report the result. Do not assume that the generated code is correct; the program must be tested.

LLM-Assisted Implementation

The generated Python implementation was then executed on the warehouse map specified in the laboratory exercise. The implementation and its results are examined in the subsequent testing tasks.

4 Task 3: Test the Generated Program

The A* program generated with the assistance of the Large Language Model was tested systematically using the cases specified in the laboratory exercise. The purpose of the tests was to verify not only that the program could produce a plausible path, but also that its behaviour was correct for different types of search problems.

Test 1: Original Warehouse

The program was first executed on the original warehouse map provided in the laboratory exercise.

The initial generated program successfully found a path from **S** to **G**. However, inspection of the returned path revealed that the starting predecessor value **None** was included in the reconstructed path. Consequently, the reported path length was one greater than the actual number of movements.

The relevant issue was caused by the following path reconstruction logic:

```
1 path = [current]
2
3 while current in came_from:
4     current = came_from[current]
5     path.append(current)
6
7 path.reverse()
```

Listing 1: Initial path reconstruction

Since the initial state had predecessor **None**, the value **None** was appended to the path.

The generated program was therefore corrected so that path reconstruction stops when the initial state is reached:

```
1 path = [current]
2
3 while came_from[current] is not None:
4     current = came_from[current]
5     path.append(current)
6
7 path.reverse()
```

Listing 2: Corrected path reconstruction

After this correction, the original warehouse produced the following result:

```
1 Solution found!
2 Path length: 40
3 States expanded: 64
4
5 Path:
6 [(1, 1), (1, 2), (1, 3), (1, 4), (1, 5),
7  (2, 5), (3, 5), (4, 5), (5, 5), (5, 6),
8  (5, 7), (5, 8), (5, 9), (5, 10), (5, 11),
9  (5, 12), (5, 13), (4, 13), (3, 13), (3, 12),
10 (3, 11), (3, 10), (3, 9), (3, 8), (3, 7),
11 (2, 7), (1, 7), (1, 8), (1, 9), (1, 10),
12 (1, 11), (1, 12), (1, 13), (1, 14), (1, 15),
13 (2, 15), (3, 15), (4, 15), (5, 15), (6, 15),
14 (7, 15)]
```

Listing 3: Result for the original warehouse

Thus, the corrected program found a solution with a path length of 40 and expanded 64 states.

Test 2: Trivial Case

A small warehouse was constructed in which the goal is immediately adjacent to the starting position:

```
#####  
#SG##  
#####
```

The expected solution consists of exactly one movement from **S** to **G**.
After correcting the path reconstruction, the program produced:

```
1  Solution found!  
2  Path length: 1  
3  States expanded: 2  
4  
5  Path:  
6  [(1, 1), (1, 2)]
```

Listing 4: Result for the trivial case

The result is correct because the goal is reached after a single movement.

Test 3: No Solution

A warehouse with an inaccessible goal was then used:

```
#####  
#S...#  
###.###  
#...#G#  
#####
```

The goal is separated from the reachable region by obstacles. The program was expected to terminate and report that no solution exists.

The program produced:

```
1  No solution found.  
2  States expanded: 9
```

Listing 5: Result for the no-solution case

The program therefore correctly terminated without entering an infinite loop and correctly reported that no path exists.

Test 4: Alternative Paths

A warehouse containing multiple possible paths from **S** to **G** was constructed:

```
#####  
#S...G#  
#.#.#.#  
#....#  
#####
```

There is a direct path along the upper row as well as longer paths through the lower part of the warehouse. Therefore, the shortest path has length 4.

The program produced:

```

1  Solution found!
2  Path length: 4
3  States expanded: 5
4
5  Path:
6  [(1, 1), (1, 2), (1, 3), (1, 4), (1, 5)]

```

Listing 6: Result for the alternative-path case

The returned path has length 4, which is the shortest possible path between the start and goal. Therefore, the test confirms that the implementation returns a shortest path for this example.

Summary of Testing

Test	Solution Found	Path Length	States Expanded
Original warehouse	Yes	40	64
Trivial case	Yes	1	2
No solution	No	–	9
Alternative paths	Yes	4	5

Table 2: Results of testing the generated A* program.

Testing Conclusion

The tests revealed a path reconstruction bug in the initial generated implementation. The bug was identified because the returned path contained `None` and the reported path length was incorrect. After correcting the reconstruction logic, the program passed all four tests: it found the original solution, correctly handled the one-step case, reported failure for an inaccessible goal, and returned the shortest path in the alternative-path case.

This demonstrates that producing a plausible path is not sufficient to establish that a search implementation is correct. Systematic tests with known expected behaviour were necessary to identify and correct the implementation error.

5 Task 4: Inspect the A* Algorithm

The generated A* program was inspected to identify how the concepts from the search formulation are implemented in the code.

Identifying the Search Concepts

Concept	Where it appears in the code
State	A state is represented by a tuple containing the row and column of the robot, such as <code>(r,c)</code> . The variables <code>start</code> , <code>goal</code> , <code>current</code> , and <code>neighbor</code> represent states.
Action	The possible actions are represented by the <code>directions</code> list: <code>(-1,0)</code> , <code>(1,0)</code> , <code>(0,-1)</code> , and <code>(0,1)</code> , corresponding to Up, Down, Left, and Right.
Transition	For a state <code>(r,c)</code> and an action <code>(dr,dc)</code> , the next state is computed using <code>new_r = r + dr</code> and <code>new_c = c + dc</code> . Boundary and obstacle checks determine whether the transition is valid.
Goal test	The goal test is performed using <code>if current == goal:</code> . When this condition is true, the path is reconstructed and returned.
$g(n)$	The cost from the start state is stored in the <code>g_cost</code> dictionary. The cost of a new state is calculated using <code>new_g = current_g + 1</code> .
$h(n)$	The heuristic is calculated by the <code>heuristic()</code> function using Manhattan distance: $h(n) = r - r_G + c - c_G .$ It is called using <code>heuristic(neighbor, goal)</code> .
$f(n)$	The A* evaluation function is explicitly calculated as <code>f = new_g + h</code> . The initial frontier entry uses <code>initial_h</code> because the initial state has $g = 0$.
Frontier	The frontier is the list <code>frontier</code> , which is maintained as a priority queue using Python's <code>heapq</code> module. Each entry contains <code>(f, g, state)</code> .
Visited states	The program does not use an explicit <code>visited</code> set. Instead, <code>g_cost</code> stores the best known cost for each state, and outdated frontier entries are ignored using <code>if current_g != g_cost[current]: continue</code> .
Path reconstruction	The <code>came_from</code> dictionary stores the predecessor of each state. The <code>reconstruct_path()</code> function follows these predecessors back from the goal until the start state is reached and then reverses the resulting list.

Table 3: Implementation of search concepts in the generated A* program.

Questions

(a) What data structure is used for the A* frontier?

The A* frontier is implemented using a Python list together with the `heapq` module. It functions as a priority queue, with each entry having the form

`(f, g, state)`

The heap maintains the entry with the smallest priority at the top.

(b) How does the program select the next state to expand?

The program uses `heapq.heappop(frontier)` to remove the entry with the smallest priority. Since the entries are stored as `(f, g, state)`, the state with the smallest $f(n)$ is selected for expansion.

(c) Where is the heuristic calculated?

The heuristic is calculated in the `heuristic()` function:

```
1 def heuristic(position, goal):
2     return abs(position[0] - goal[0]) + abs(position[1] - goal[1])
```

Listing 7: Manhattan-distance heuristic

It is called when a neighboring state is generated:

```
1 h = heuristic(neighbor, goal)
```

(d) Does the program explicitly calculate $f(n) = g(n) + h(n)$?

Yes. For every newly discovered or improved neighbor, the program first calculates its path cost using

```
1 new_g = current_g + 1
```

and then calculates the heuristic:

```
1 h = heuristic(neighbor, goal)
```

The two values are explicitly combined:

```
1 f = new_g + h
```

The resulting value is then inserted into the frontier.

(e) How does the program prevent unnecessary repeated exploration?

The program uses the `g_cost` dictionary to store the best known cost for each state. A neighboring state is added to the frontier only when the newly calculated cost is better than its previously known cost:

```
1 if new_g < g_cost.get(neighbor, float("inf")):
```

In addition, the program checks for outdated entries when removing a state from the frontier:

```
1 if current_g != g_cost[current]:
2     continue
```

Thus, an older frontier entry is skipped if a better path to the same state has already been found. The implementation therefore prevents unnecessary repeated expansion without using an explicit `visited` set.

Think About It

A program producing a plausible path is not sufficient to establish that the search algorithm is correct. Testing should deliberately include cases where the expected behaviour is known.

In this experiment, systematic testing revealed that the initial generated program contained a path reconstruction error. Although the program produced a plausible path, the returned path incorrectly contained `None`, resulting in an incorrect path length. The trivial, no-solution, and alternative-path tests were then used to verify that the corrected program behaved as expected.

This demonstrates the principle:

Working output $\neq$ validated algorithm

Testing with known expected behaviour is therefore an important part of AI engineering.

6 Task 5: Compare A* with Blind Search

To compare informed and blind search, a Breadth-First Search (BFS) version of the agent was implemented. The same warehouse map, state representation, actions, transition rules, and goal condition were used for both BFS and A*. The warehouse was not changed.

BFS Implementation

BFS uses a FIFO queue as its frontier. When a state is expanded, all valid neighboring states that have not already been discovered are added to the queue. Since every movement has the same cost of 1, BFS can find a shortest path in this problem.

Comparison of BFS and A*

Measure	BFS	A*
Solution found	Yes	Yes
Path length	40	40
States expanded	64	64

Table 4: Comparison of BFS and A* on the original warehouse.

(a) Did both algorithms find a solution?

Yes. Both BFS and A* successfully found a path from S to G.

(b) Did they find paths of the same length?

Yes. Both algorithms found a path of length 40.

(c) Which algorithm expanded fewer states?

Neither algorithm expanded fewer states in this experiment. Both BFS and A* expanded 64 states.

(d) Why might A* expand fewer states?

A* uses the heuristic $h(n)$ to estimate the remaining cost to the goal. Therefore, instead of exploring states only according to their distance from the start, A* can prioritize states that appear to be closer to the goal.

BFS does not use any information about the goal when selecting the next state to expand. Consequently, on a problem where the heuristic gives useful guidance, A* may expand fewer states than BFS.

In this particular warehouse, however, both algorithms expanded the same number of states. Therefore, the heuristic did not reduce the number of expanded states for this specific map under the implemented tie-breaking and expansion rules.

Observation

Both algorithms produced an optimal path of length 40. Since every movement has equal cost, BFS is guaranteed to find a shortest path. A* also found the same shortest path using the Manhattan-distance heuristic. The experiment shows that A* is not automatically better than BFS in terms of the number of states expanded; its advantage depends on how effectively the heuristic guides the search.

Think About It

The information used to decide which state to search next differs between the two algorithms. BFS uses only the order in which states are discovered, so it explores states in FIFO order without using information about their distance from the goal.

A* uses both the cost accumulated from the start, $g(n)$, and the heuristic estimate of the remaining cost, $h(n)$. It combines them as

$$f(n) = g(n) + h(n),$$

and expands the state with the smallest $f(n)$. Thus, A* uses information about both the path already travelled and the estimated distance to the goal.

7 Task 6: Investigate the Heuristic

The laboratory uses the Manhattan-distance heuristic

$$h(n) = |r - r_G| + |c - c_G|.$$

This heuristic is appropriate for the warehouse because the robot can move only horizontally or vertically, and every movement has a cost of 1. The Manhattan distance represents the minimum number of horizontal and vertical movements required to reach the goal when obstacles are ignored.

The behaviour of A* was investigated experimentally by replacing the Manhattan heuristic with three alternatives: $h(n) = 0$, Euclidean distance, and twice the Manhattan distance.

Experimental Results

Heuristic	Solution Found	Path Length	States Expanded
Manhattan	Yes	40	64
$h(n) = 0$	Yes	40	64
Euclidean	Yes	40	64
2× Manhattan	Yes	40	67

Table 5: Results for different heuristic functions.

1. Replacing the Heuristic with $h(n) = 0$

When $h(n) = 0$, the evaluation function becomes

$$f(n) = g(n).$$

The heuristic therefore provides no information about the estimated distance to the goal. The program still found a solution of length 40 and expanded 64 states.

2. Replacing the Heuristic with Euclidean Distance

The Euclidean heuristic is

$$h(n) = \sqrt{(r - r_G)^2 + (c - c_G)^2}.$$

The program found a solution of length 40 and expanded 64 states. Thus, for this particular warehouse, the Euclidean heuristic produced the same measured result as the Manhattan heuristic.

3. Multiplying the Manhattan Heuristic by 2

The modified heuristic is

$$h(n) = 2(|r - r_G| + |c - c_G|).$$

The program found a solution of length 40 but expanded 67 states. Thus, although the returned path was still of the same length in this experiment, the search behaviour differed from the original Manhattan heuristic.

Comparison

All four heuristic versions found a solution with path length 40. Manhattan distance, $h(n) = 0$, and Euclidean distance each expanded 64 states, while $2 \times$ Manhattan expanded 67 states.

Therefore, for this particular warehouse and implementation, changing the heuristic did not change the path length. However, it did affect the number of states expanded when the Manhattan heuristic was multiplied by 2.

Think About It

The lecture introduced admissibility as

$$h(n) \leq h^*(n),$$

where $h^*(n)$ is the true minimum cost from state n to the goal.

For this warehouse, Manhattan distance is an appropriate lower bound because the robot can move only horizontally and vertically, with each movement costing 1. Obstacles can only increase the actual path cost, so the Manhattan distance does not overestimate the cost of reaching the goal.

The heuristic $h(n) = 0$ also satisfies the admissibility condition because it never overestimates the remaining cost. However, it provides no information about which states are closer to the goal.

Euclidean distance also provides a lower bound on the number of horizontal and vertical movements required to reach the goal. In this experiment, it produced the same path length and number of expanded states as Manhattan distance.

The heuristic $2 \times$ Manhattan can overestimate the actual remaining cost and is therefore not generally admissible. In this experiment it still found a path of length 40, but it expanded 67 states instead of 64. Thus, the experiment shows that an aggressive heuristic can change the search behaviour even when the resulting path happens to remain optimal for a particular problem.

Overall, the experiment demonstrates that the behaviour of A* depends on the heuristic used. The experimental results also show that obtaining an optimal path in one particular test case does not by itself establish that a heuristic is admissible or that optimality is guaranteed in general.

8 Task 7: Evaluate the LLM-Generated Agent

This task evaluates the use of the Large Language Model as an engineering assistant during the implementation of the A* search agent. The process was divided into the parts that were designed independently, suggested by the LLM, accepted, modified, and tested.

1. What parts of the generated code were correct immediately?

Most of the main A* components generated by the LLM were correct immediately. The program correctly represented states as row-column coordinates, represented the warehouse as a grid, generated the four possible movement actions, checked grid boundaries and obstacles, calculated $g(n)$ and the Manhattan heuristic $h(n)$, and used $f(n) = g(n) + h(n)$ to prioritize states.

The use of a priority queue with `heapq`, the `g_cost` dictionary, and the `came_from` dictionary were also appropriate for implementing A* search.

2. Did you find any bugs or design problems?

Yes. The initial implementation contained a bug in the path reconstruction function. The starting state had `None` as its predecessor, but the original reconstruction loop continued until the current state was no longer present in `came_from`. As a result, `None` was included in the returned path.

This caused the initially reported path length to be one greater than the actual number of movements.

3. How did you discover those problems?

The problem was discovered by inspecting the output of the original warehouse test. Although the program reported that a solution had been found, the reconstructed path contained `None` at its beginning.

This showed that the program was producing a plausible result but that the implementation was not completely correct. Additional systematic tests were then used to verify the corrected implementation, including a trivial case, a no-solution case, and a case with alternative paths.

4. Did the LLM use terminology or data structures that you did not understand?

The LLM used standard A* terminology and data structures such as the frontier, priority queue, heuristic, $g(n)$, $h(n)$, $f(n)$, `g_cost`, and `came_from`. These concepts were examined in the implementation rather than being accepted without understanding their purpose.

In particular, inspecting the code made it clear how the priority queue selects the next state and how `g_cost` and the stale-entry check prevent unnecessary repeated exploration.

5. Did you modify the LLM-generated code?

Yes. The path reconstruction function was modified after the testing revealed the `None` value in the returned path.

The original logic was changed so that reconstruction stops when the initial state is reached rather than following its `None` predecessor. The rest of the main A* structure was retained.

6. Which tests were most useful?

The original warehouse test was particularly useful because it revealed the path reconstruction bug even though the search itself successfully found a path.

The no-solution test was useful for verifying that the program terminates correctly when the goal cannot be reached. The trivial case verified the basic one-step behaviour, while the alternative-path case tested whether the algorithm returned a shortest path when multiple paths were available.

Together, these tests checked different aspects of the implementation rather than only checking whether one example produced a path.

7. Could you have trusted the program without testing it?

No. The initial program produced a plausible solution, but testing revealed that its reported path was incorrect because of the path reconstruction bug.

Therefore, a program that appears to work on one example cannot be assumed to implement the intended algorithm correctly. Systematic testing with known expected behaviour was necessary to validate the implementation.

8. What did you understand about A* that you did not understand before implementing it?

Implementing and inspecting the algorithm made the relationship between the different components of A* clearer. In particular, it showed how $g(n)$ represents the cost already incurred, $h(n)$ estimates the remaining cost, and

$$f(n) = g(n) + h(n)$$

determines which state is prioritised for expansion.

The experiments also demonstrated that the behaviour of A* depends on the heuristic. Different heuristics can change the number of states expanded, while an inadmissible heuristic is not guaranteed to preserve optimality in general.

What Was Designed, Suggested, Accepted, Changed, and Tested

Category	Description
Designed by us	The search formulation, state representation, actions, transition rules, goal condition, path reconstruction requirement, and use of Manhattan distance were specified before asking the LLM for code.
Suggested by the LLM	The concrete Python implementation of A*, including the priority queue, <code>g_cost</code> , <code>came_from</code> , and the search loop.
Accepted	The main A* search structure and its representation of states, actions, costs, heuristic, frontier, and predecessor information were retained after inspection.
Changed	The path reconstruction logic was modified after the initial test revealed that <code>None</code> was incorrectly included in the path.
Tested	The corrected implementation was tested on the original warehouse, trivial case, no-solution case, and alternative-path case. It was also compared with BFS and tested with different heuristic functions.

Table 6: Distinction between design, LLM contribution, modification, and testing.

Conclusion

The LLM was useful for producing the initial implementation and providing the basic structure of the A* agent. However, the generated program could not simply be assumed to be correct. Testing revealed an implementation error that was not obvious from the initial successful search result.

The experiment therefore demonstrated that an LLM can be used as an engineering assistant, but the generated code must be understood, inspected, modified when necessary, and systematically tested before it is considered reliable.

9 Final Reflection

1. Why is it important to formulate the search problem before writing the search algorithm?

Formulating the search problem first identifies the states, actions, transitions, initial state, goal condition, and costs of the problem. This separates the problem itself from the algorithm used to solve it. In this experiment, defining the warehouse in terms of grid positions, valid movements, the start and goal states, and movement costs made it possible to implement and evaluate A* systematically. Without a clear formulation, it would be difficult to determine whether an algorithm is solving the intended problem correctly.

2. In what sense is A* an “informed” search algorithm?

A* is an informed search algorithm because it uses additional information about the problem in the form of a heuristic. It evaluates a state using

$$f(n) = g(n) + h(n),$$

where $g(n)$ is the cost already incurred and $h(n)$ estimates the remaining cost to the goal. In this experiment, the Manhattan-distance heuristic provided information about the location of the goal and helped determine which states should be considered next. In contrast, a blind search such as BFS does not use an estimate of the remaining distance to the goal.

3. Why does the choice of heuristic matter?

The heuristic affects how A* prioritises states for exploration. A useful heuristic can provide information about which states are promising while an admissible heuristic preserves the optimality guarantee of A*. In the experiments, Manhattan distance, zero heuristic, and Euclidean distance all found paths of length 40 and expanded 64 states, while $2\times$ Manhattan expanded 67 states. The experiment therefore showed that changing the heuristic can change the behaviour of the search, while an inadmissible heuristic is not guaranteed to preserve optimality in general.

4. What did the LLM contribute to the engineering process?

The LLM contributed the initial concrete Python implementation of the A* agent based on the search design that had been specified beforehand. It provided the structure for the priority queue, cost tracking, heuristic calculation, successor generation, and path reconstruction. However, the LLM was used as an engineering assistant rather than as an unquestioned source of correct code. The generated implementation was inspected and tested, and the testing process revealed a path reconstruction bug that was then corrected.

5. What could go wrong if an engineer simply accepted LLM-generated code without testing it?

An LLM-generated program can contain implementation errors even when its output appears plausible. In this experiment, the initial A* program successfully found a path but incorrectly included `None` in the reconstructed path, causing the reported path length to be incorrect. Without testing, this error could have gone unnoticed. More generally, untested generated code may fail on edge cases, incorrectly handle situations such as unreachable goals, or implement an algorithm differently from what was intended. Therefore, generated code must be inspected and systematically tested before being relied upon.